

Gerrymandering Metrics

```
library(tidyverse)
```

```
— Attaching core tidyverse packages ————— tidyverse 2.0.0
—
✓ dplyr      1.1.4    ✓ readr      2.1.5
✓ forcats    1.0.1    ✓ stringr    1.5.2
✓ ggplot2    4.0.0    ✓ tibble     3.3.0
✓ lubridate  1.9.4    ✓ tidyr      1.3.1
✓ purrr      1.1.0
— Conflicts ————— tidyverse_conflicts()
—
* dplyr::filter() masks stats::filter()
* dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors
```

```
library(readr)
library(sf)
```

Linking to GEOS 3.13.1, GDAL 3.11.0, PROJ 9.6.0; sf_use_s2() is TRUE

```
library(tigris)
```

To enable caching of data, set `options(tigris\_use\_cache = TRUE)`
in your R script or .Rprofile.

```
library(dplyr)
library(ggplot2)
library(stats)
precinct_2024_general_election <- read.csv("data/g24_sov_by_g24_svprec.csv")

simpler_precinct <- precinct_2024_general_election |>
  select("COUNTY", "TOTVOTE", "CDDIST", "ASSDEM01", "ASSDEM02", "ASSREP01",
         "ASSREP02", "CNGDEM01", "CNGDEM02", "CNGREP01", "CNGREP02",
         "PRSDEM01", "PRSREP01", "SENDEM01", "SENDEM02",
         "SENREP01", "SENREP02", "USSDEM01", "USSREP01") |>
  filter(
```

```

    !if_any(
      .cols = -COUNTY,
      .fns = ~ . == "***"
    ) |>
      mutate(
        across(
          .cols = -COUNTY,
          .fns = ~ as.integer(as.character(.))
        )
      )
votes_sr <- read_csv("data/state_g24_sov_data_by_g24_srprec.csv")

```

Rows: 25245 Columns: 76

— Column specification

Delimiter: ","

chr (49): FIPS, SRPREC, ELECTION, SRPREC_KEY, GEO_TYPE, ASSAIP01, ASSDEM01, ...

dbl (27): COUNTY, ADDIST, CDDIST, SDDIST, BEDIST, TOTREG, DEMREG, REPREG, AI...

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
block_corr <- read_csv("data/state_g24_sr_blk_map.csv")
```

Rows: 421137 Columns: 14

— Column specification

Delimiter: ","

chr (6): BLOCK_KEY, FIPS, SRPREC, SRPREC_KEY, ELECTION, GEO_TYPE

dbl (8): COUNTY, TRACT, BLOCK, BLKREG, SRTOTREG, PCTSRPREC, BLKTOTREG, PCTBLK

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
new_district_map <- read_csv("data/ab604.csv")
```

Rows: 519722 Columns: 2

— Column specification

Delimiter: ","

```
chr (2): 060650444033050, 25
```

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
votes_sr <- votes_sr |>
  select(SRPREC_KEY, CNGDEM01, CNGREP01)
block_corr <- block_corr |>
  select(BLOCK_KEY, SRPREC_KEY, PCTSRPREC)
new_district_map <- read_csv(
  "data/ab604.csv",
  col_names = c("BLOCK_KEY", "NEW_CD"),
) |>
  mutate(BLOCK_KEY = as.character(BLOCK_KEY)) |>
  select(BLOCK_KEY, NEW_CD)
```

```
Rows: 519723 Columns: 2
— Column specification
```

```
Delimiter: ","
chr (2): BLOCK_KEY, NEW_CD
```

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
allocated_data <- block_corr |>
  left_join(votes_sr, by = "SRPREC_KEY") |>
  mutate(
    CNGDEM01 = as.numeric(CNGDEM01),
    CNGREP01 = as.numeric(CNGREP01),

    PCTSRPREC = as.numeric(PCTSRPREC)
  ) |>

  mutate(
    D_Allocated = CNGDEM01 * (PCTSRPREC / 100),
    R_Allocated = CNGREP01 * (PCTSRPREC / 100)
  )
```

Warning: There were 2 warnings in `mutate()`.
The first warning was:
i In argument: `CNGDEM01 = as.numeric(CNGDEM01)`.

Caused by warning:
! NAs introduced by coercion
i Run `dplyr::last\_dplyr\_warnings()` to see the 1 remaining warning.

```
final_results_ab604 <- allocated_data |>
  left_join(new_district_map, by = "BLOCK_KEY") |>
  filter(!is.na(NEW_CD)) |>
  group_by(NEW_CD) |>

  summarise(
    New_D_Votes = sum(D_Allocated, na.rm = TRUE),
    New_R_Votes = sum(R_Allocated, na.rm = TRUE),
    .groups = 'drop'
  ) |>

  mutate(
    New_Total = New_D_Votes + New_R_Votes,
    New_D_Pct = New_D_Votes / New_Total,
    New_Winner = ifelse(New_D_Votes > New_R_Votes, "DEM", "REP")
  )
```

2024 Election Results and the 2024 District Map

```
district_data <- simpler_precinct |>
  group_by(CDDIST) |>
  summarise(
    D_Votes = sum(CNGDEM01),
    R_Votes = sum(CNGREP01),
    .groups = 'drop'
  ) |>

  mutate(
    Two_Party_Total = D_Votes + R_Votes,
    D_Pct = D_Votes / Two_Party_Total
  ) |>
  filter(Two_Party_Total > 0)
print(head(district_data))
```

```
# A tibble: 6 × 5
  CDDIST D_Votes R_Votes Two_Party_Total D_Pct
  <int>   <int>   <int>         <int> <dbl>
1     0 52843210 34247705         87090915 0.607
2     1  110472  208150          318622 0.347
3     2  272384  106407          378791 0.719
4     3  187960  233895          421855 0.446
```

5	4	227321	114644	341965	0.665
6	5	134467	214223	348690	0.386

```
calculate_wasted_votes <- function(party_A_votes, party_B_votes) {
  total_votes <- party_A_votes + party_B_votes
  votes_to_win <- floor(total_votes / 2) + 1

  winner <- party_A_votes > party_B_votes
  wasted_A_winner <- party_A_votes - votes_to_win
  wasted_A_loser <- party_A_votes
  wasted_votes_A <- ifelse(winner, wasted_A_winner, wasted_A_loser)
  return(wasted_votes_A)
}
```

```
mean_d_pct <- mean(district_data$D_Pct)
median_d_pct <- median(district_data$D_Pct)
mean_median_score <- (mean_d_pct - median_d_pct) * 100

district_wasted_votes <- district_data |>
  mutate(
    D_Wasted = calculate_wasted_votes(D_Votes, R_Votes),
    R_Wasted = calculate_wasted_votes(R_Votes, D_Votes)
  )

wasted_votes_summary <- district_wasted_votes |>
  summarise(
    Total_D_Wasted = sum(D_Wasted),
    Total_R_Wasted = sum(R_Wasted),
    Total_Statewide_Votes = sum(Two_Party_Total)
  )

TDW <- wasted_votes_summary$Total_D_Wasted
TRW <- wasted_votes_summary$Total_R_Wasted
TVotes <- wasted_votes_summary$Total_Statewide_Votes

efficiency_gap <- ((TRW - TDW) / TVotes) * 100

print("Mean median score")
```

```
[1] "Mean median score"
```

```
print(mean_median_score)
```

```
[1] -1.163046
```

```
print("Efficiency Gap")
```

```
[1] "Efficiency Gap"
```

```
print(efficiency_gap)
```

```
[1] 25.96726
```

2024 Election Results and the proposed 2025 District Map

```
mean_d_pct_new <- mean(final_results_ab604$New_D_Pct, na.rm = TRUE)
median_d_pct_new <- median(final_results_ab604$New_D_Pct, na.rm = TRUE)
mean_median_score_new <- (mean_d_pct_new - median_d_pct_new) * 100
```

```
district_wasted_votes_new <- final_results_ab604 |>
  mutate(
    D_Wasted = calculate_wasted_votes(New_D_Votes, New_R_Votes),
    R_Wasted = calculate_wasted_votes(New_R_Votes, New_D_Votes)
  )
```

```
wasted_votes_summary_new <- district_wasted_votes_new |>
  summarise(
    Total_D_Wasted = sum(D_Wasted, na.rm = TRUE),
    Total_R_Wasted = sum(R_Wasted, na.rm = TRUE),
    Total_Statewide_Votes = sum(New_Total, na.rm = TRUE)
  )
```

```
TDW_new <- wasted_votes_summary_new$Total_D_Wasted
TRW_new <- wasted_votes_summary_new$Total_R_Wasted
TVotes_new <- wasted_votes_summary_new$Total_Statewide_Votes
```

```
efficiency_gap_new <- ((TRW_new - TDW_new) / TVotes_new) * 100
```

```
print("New Mean median score")
```

```
[1] "New Mean median score"
```

```
print(mean_median_score_new)
```

```
[1] 3.048643
```

```
print("New Efficiency Gap")
```

```
[1] "New Efficiency Gap"
```

```
print(efficiency_gap_new)
```

```
[1] 20.15427
```